

TCGA KIRC ccRCC Mutation Database

Project Documentation & Reproduction Guide

BINF6970 — Databases for Bioinformatics | Georgetown University | Spring 2026

1. Project Overview

This project builds a reproducible, fully normalized relational database from TCGA (The Cancer Genome Atlas) clinical and somatic mutation data for kidney renal clear cell carcinoma (CCRCC/KIRC), selects biologically significant single nucleotide polymorphisms (SNPs) based on functional impact and recurrence criteria, and imports them into a Neo4j graph database enriched with protein chemistry and interaction data from four public REST APIs.

The database enables downstream analysis of somatic mutation patterns in 512 CCRCC tumor samples, identification of recurrently mutated genes, and characterization of the chemical consequences of amino acid substitutions caused by missense mutations. The project demonstrates an end-to-end bioinformatics data engineering pipeline from raw cBioPortal export files to an enriched, queryable graph database.

Component	Description
Relational database	MySQL 8.0 with 7 normalized tables, 512 samples, 31,073 mutations
Graph database	Neo4j with 5 SNP nodes, 5 Gene nodes, 12 Sample nodes, PPI network
Data cleaning	Python pipeline normalizing source files to 1NF–5NF
SNP selection	SQL queries selecting recurrent, double-damaging missense SNPs
API enrichment	UniProt, dbSNP, ClinVar, STRING, Grantham score computation
Population context	All of Us Researcher Workbench variant and EHR analysis

2. Data Sources

All source data was downloaded from cBioPortal (<https://www.cbioportal.org>) for the study: TCGA Kidney Renal Clear Cell Carcinoma PanCancer Atlas 2018 (study ID: kirc_tcga_pan_can_atlas_2018). The data is publicly available and does not require access approval.

File	Type	Rows	Columns	Description
data_clinical_sample.txt	TSV (cBioPortal format)	512	19	Sample-level clinical data including grade, MSI scores, TMB, and tissue collection metadata
data_mutations.txt	MAF (TCGA format)	31,073	114	Somatic mutation annotation file produced by the GDC pipeline using GRCh37 reference genome

The cBioPortal clinical sample file uses a four-row metadata header before the data header row. The mutation file conforms to the TCGA Mutation Annotation Format (MAF) and was produced using an ensemble of five somatic variant callers: SomaticSniper, RADIA, MuTect, MUSE, and VarScan2. All mutations are somatic calls with matched normal tissue.

3. Data Cleaning Decisions

All cleaning was performed by script 01_clean_tcgaccrcc.py. Each decision is documented below with its rationale.

3.1 Clinical Sample File

Header row skipping

The first four rows of the file are cBioPortal annotation rows describing column semantics (human label, description, data type, display priority). They are not data rows and were skipped unconditionally using `skiprows=4`. Retaining them would cause all columns to be parsed as strings with values offset by four rows.

Blank string normalization

Pandas reads blank cells as empty strings by default. These were replaced globally with `pd.NA` at load time to ensure consistent NULL semantics across all subsequent operations. Without this step, `isnull()` and `notna()` checks would fail silently on blank-string cells.

Boolean indicator encoding

TISSUE_PROSPECTIVE_COLLECTION_INDICATOR and TISSUE_RETROSPECTIVE_COLLECTION_INDICATOR contained Yes, No, and blank string values. These were mapped to 1, 0, and NULL. Note: although the cleaning script performed this mapping, phpMyAdmin type inference during CSV import stored these columns as VARCHAR in the live database rather than TINYINT(1). This is documented as a known deviation in Section 7.

Numeric coercion

ANEUPLOIDY_SCORE, MSI_SCORE_MANTIS, MSI_SENSOR_SCORE, TMB_NONSYNONYMOUS, and TBL_SCORE were coerced to numeric types using `pd.to_numeric` with `errors='coerce'`. Non-parseable values were converted to NaN. cBioPortal exports all columns as strings, so this step was required to enable numeric operations. Similar to the boolean columns, phpMyAdmin type inference stored most of these as VARCHAR in the live database.

Duplicate and FK validation

SAMPLE_ID values were checked for duplicates; no duplicates were found. Each SAMPLE_ID was validated to begin with its PATIENT_ID using a row-wise apply, consistent with the TCGA barcode convention where the patient code is embedded as a prefix of the sample barcode (e.g. TCGA-3Z-A93Z-01 derives from patient TCGA-3Z-A93Z). No FK violations were found.

3.2 Mutation File

Column selection — 88 of 114 columns dropped

Category	Examples	Reason
Pipeline metadata	Center, Sequencer, BAM_File, Sequencing_Phase	Sequencing center configuration. Not analytically relevant.
Redundant alleles	Tumor_Seq_Allele1, Validation alleles	Tumor_Seq_Allele1 equals Reference_Allele in all rows (no LOH). Validation alleles are uniformly empty.
Population frequencies	AA_MAF, EUR_MAF, ExAC_AF and subpopulations	Germline population frequencies from 1000G and ExAC. Not relevant for somatic analysis.
Redundant VEP flags	CANONICAL, BIOTYPE, SYMBOL_SOURCE, PICK	Transcript selection logic flags. Canonical consequence already captured in Variant_Classification and Transcript_ID.
Multi-value compound fields	all_effects, DOMAINS, PUBMED, COSMIC	Pipe-delimited multi-value cells violating 1NF. Canonical consequence captured in single-value fields.

Dot-null sentinel replacement

cBioPortal MAF exports use a period (.) as a NULL sentinel in string columns including dbSNP_RS, HGVSc, HGVSp_Short, Transcript_ID, PolyPhen, SIFT, and Chromosome. These were replaced with `pd.NA`. Retaining the period as a value would cause IS NULL queries to miss affected rows.

Hotspot encoding correction

The Hotspot column contained values of 0 and NaN only. The original script used a lambda checking `x == 1` which, since no values equalled 1, caused every row to evaluate to False and

write the string 'False' into the TSV rather than the integer 0. This was corrected to `fillna(0).astype(int)`.

Integer float encoding

Pandas reads integer columns containing any NaN values as float64, appending .0 to every value. MySQL rejected inserts of 400.0 into INT columns. All integer columns were cast to pandas nullable Int64 dtype which preserves integer representation while supporting NaN, writing clean integers and empty strings for nulls.

TSV quoting

All output TSVs were written with `csv.QUOTE_ALL (quoting=1)`, wrapping every field in double quotes. This was necessary because HGVS_p_Short values containing asterisks (e.g. p.*373*) were misinterpreted by phpMyAdmin as wildcard characters, causing column count errors on import.

Orphan and duplicate mutation removal

Mutations were validated against the sample table; any referencing an unknown sample barcode were excluded. Mutations were also deduplicated on the composite key of sample, chromosome, position, reference allele, and alternate allele. No orphans or duplicates were found in this dataset.

4. Database Design Decisions

4.1 Why MySQL with a 7-table normalized schema

The source data is rectangular and relational in nature — samples reference patients, mutations reference samples and genes. A relational database with foreign key enforcement is the appropriate choice because it guarantees referential integrity (no mutation can reference a non-existent gene), enables complex multi-table joins for analytical queries, and allows the schema to be version-controlled and reproduced from a SQL script.

4.2 Surrogate key for mutation table

A surrogate integer `mutation_id` was used rather than a composite natural key across chromosome, position, sample, and allele. The natural key would span five columns and be unwieldy as a foreign key target in any future junction table. The surrogate key satisfies 1NF while keeping join performance practical.

4.3 Why Neo4j for the SNP graph

A graph database is appropriate for the SNP enrichment step because the relationships between entities are the primary analytical object: which samples share a mutation, which genes interact at the protein level, what is the chemical distance between mutated amino acids. These queries are natural graph traversals that would require complex multi-join SQL but are expressed as simple Cypher path queries. Neo4j also allows heterogeneous node and relationship properties, enabling API-enriched attributes (Grantham score, ClinVar significance, STRING interaction scores) to be added without schema migration.

4.4 SNP selection criteria

SNPs were selected from the mutation table using the following mandatory criteria applied via SQL SELECT with HAVING:

- filter_status = 'PASS' — removes low-quality variant calls flagged by the ensemble callers
- variant_type = 'SNP' — excludes insertions and deletions; focus is on single nucleotide changes
- variant_classification = 'Missense_Mutation' — protein-altering changes only
- polyphen LIKE 'probably_damaging%' — PolyPhen-2 predicts functional damage
- sift LIKE 'deleterious%' — SIFT independently predicts functional damage
- dbsnp_rs IS NOT NULL — variant must be registered in dbSNP for population context lookup
- HAVING COUNT(DISTINCT sample_id) >= 2 — same amino acid change in at least 2 independent patients

The top 5 results by n_samples and mean_vaf were selected, giving priority to recurrent mutations with high tumor allele frequency (suggesting clonal rather than subclonal events).

5. Normalization Decisions (1NF–5NF)

Normal Form	Rule Applied	Implementation
1NF	Atomic values, no repeating groups, unique PKs	Multi-value columns (all_effects, DOMAINS) dropped. Surrogate mutation_id assigned.
2NF	No partial dependencies on composite keys	cancer_type and tissue_source_site extracted from sample. Attributes depend on full PK.
3NF	No transitive dependencies	All sample attributes depend directly on sample_id. No non-key attribute determines another.
4NF	No multi-valued facts in one row	all_effects dropped. Canonical single-transcript consequence stored atomically.

Normal Form	Rule Applied	Implementation
5NF	No join dependencies	gene table: hugo_symbol depends on entrez_gene_id, not on mutation. variant_class table: impact and variant_type are fixed properties of the classification.

All deviations from the intended normal forms caused by phpMyAdmin import behavior are documented in Section 7.

6. Relationships and Constraints

Relationship	Type	Constraint	Notes
patient → sample	1:many	FK patient_id ON UPDATE CASCADE ON DELETE RESTRICT	One patient can have multiple samples
tissue_source_site → sample	1:many	FK tss_code ON UPDATE CASCADE ON DELETE RESTRICT	One institution contributes multiple samples
cancer_type → sample	1:many	FK oncotree_code ON UPDATE CASCADE ON DELETE RESTRICT	One cancer type classifies multiple samples; single value in this cohort
sample → mutation	1:many	FK sample_id ON UPDATE CASCADE ON DELETE RESTRICT	One sample contains many mutations
gene → mutation	1:many	FK entrez_gene_id ON UPDATE CASCADE ON DELETE RESTRICT	One gene harbors many mutations; entrez_gene_id=0 is placeholder for unresolved genes
variant_class → mutation	1:many	FK variant_classification ON UPDATE CASCADE ON DELETE RESTRICT	One classification applies to many mutations

All foreign keys use ON UPDATE CASCADE so that correcting a primary key value (e.g. fixing an Entrez Gene ID) automatically propagates to all referencing rows. ON DELETE RESTRICT prevents accidental deletion of parent records that have dependent children.

NULL is permitted on `entrez_gene_id` in the mutation table to accommodate the small number of mutations referencing genes with no Entrez ID at all. `entrez_gene_id = 0` is used for the 744 genes with IDs recorded as 0 in the source MAF, mapping to a documented placeholder row in the gene table.

7. Deviations from Intended Schema

The following columns have types in the live database that differ from the intended DDL due to phpMyAdmin type inference during CSV import. The intended types are documented in `01_create_schema.sql` comments. These deviations do not affect data correctness but would need to be resolved before numeric aggregation queries are run on those columns.

Table	Column	Intended Type	Live Type	Cause
sample	tissue_prospective	TINYINT(1)	VARCHAR(18)	phpMyAdmin inferred string type from CSV
sample	tissue_retrospective	TINYINT(1)	VARCHAR(20)	phpMyAdmin inferred string type from CSV
sample	aneuploidy_score	SMALLINT	VARCHAR(5)	phpMyAdmin inferred string type from CSV
sample	msi_score_mantis	DECIMAL(8,4)	VARCHAR(12)	phpMyAdmin inferred string type from CSV
sample	msi_sensor_score	DECIMAL(8,4)	VARCHAR(12)	phpMyAdmin inferred string type from CSV
sample	tmb_nonsynonymous	DECIMAL(10,6)	VARCHAR(12)	phpMyAdmin inferred string type from CSV
gene	hugo_symbol	VARCHAR(50)	VARCHAR(100)	phpMyAdmin widened column on import
mutation	reference_allele	VARCHAR(500)	TEXT	Changed manually during import troubleshooting
mutation	protein_position	INT UNSIGNED	VARCHAR(10)	Empty cell handling failure with INT in phpMyAdmin

To fix these deviations in the live database, run the ALTER TABLE statements documented in the project discussion log or re-import using LOAD DATA INFILE which respects the schema types.

8. Data Dictionary

Full data dictionary is provided in docs/data_dictionary.md. Key tables are summarized below.

8.1 MySQL Tables

Table	PK	FK References	Row Count	Purpose
patient	patient_id	—	512	Identity record for each TCGA patient
tissue_source_site	tss_code	—	20	Institution name lookup; 2NF decomposition from sample
cancer_type	oncotree_code	—	1	Cancer classification lookup; 2NF decomposition from sample
sample	sample_id	patient, cancer_type, tissue_source_site	512	Tumor sample with clinical and genomic biomarkers
gene	entrez_gene_id	—	~3,800	Gene symbol lookup; 5NF decomposition from mutation
variant_class	variant_classification	—	17	Variant type and impact lookup; 5NF decomposition from mutation
mutation	mutation_id (surrogate)	sample, gene, variant_class	31,073	Somatic mutation fact table

8.2 Neo4j Node and Relationship Types

Type	Label	Key Properties	Count
Node	Gene	entrez_gene_id, hugo_symbol, pathway, role, UniProt data	5
Node	SNP	mutation_id, hgvs_short, grantham_score, polyphen, sift, ClinVar data	5
Node	Sample	sample_id, patient_id	12
Relationship	HARBORS	Gene → SNP	5
Relationship	IN_GENE	SNP → Gene (back-reference)	5
Relationship	HAS_MUTATION	Sample → SNP; property: vaf	12
Relationship	INTERACTS_WITH	Gene — Gene; property: string_score	from STRING API

9. Script Map

Order	Script	Input	Output	Purpose
1	scripts/01_clean_tcga_ccrcc.py	data/raw/*.txt	data/cleaned/*.tsv	Clean and normalize source data into 7 TSV files
2	sql/01_create_schema.sql	Empty MySQL instance	tcga_ccrcc database with 7 empty tables	Create all tables with PKs, FKs, and indexes
3	Manual phpMyAdmin import + SQL fixes	data/cleaned/*.tsv	Populated MySQL tables	Load TSVs in FK dependency order; apply 4 post-import SQL corrections
4	scripts/02_snp_selection_queries.sql	Populated MySQL	Query results (export as CSV)	Select 5 SNPs of interest by recurrence and functional criteria
5	scripts/03_neo4j_import.cypher	Neo4j Browser	Gene, SNP, Sample nodes and relationships	Build base graph from 5 selected SNPs
6	scripts/04_api_enrichment.py	Running Neo4j + internet	Enriched Neo4j node properties	Add UniProt, dbSNP, ClinVar, STRING, and Grantham data to graph

10. Reproduction Instructions

10.1 Requirements

Software	Version	Purpose
Python	3.10+	Data cleaning and API enrichment scripts
pandas	2.0+	Data manipulation in cleaning script
requests	2.28+	REST API calls in enrichment script
neo4j (Python driver)	5.x	Neo4j connection in enrichment script
MySQL	8.0+	Relational database engine
phpMyAdmin	5.x	Database management interface (optional; can use MySQL CLI)
Neo4j Desktop	5.x	Graph database engine

Install Python dependencies:

```
pip install pandas openpyxl requests neo4j
```

10.2 Option A — From SQL dump (fastest)

1. Open MySQL and run:

```
mysql -u root -p < sql/database_dump.sql
```

2. Skip to Step 5 below.

10.3 Option B — From scratch

3. **Step 1** — Run the cleaning script:

```
python scripts/01_clean_tcga_ccrcc.py
```

Output: 7 TSV files in data/cleaned/

4. **Step 2** — Create the MySQL schema:

```
mysql -u root -p < sql/01_create_schema.sql
```

5. **Step 3** — Import TSVs via phpMyAdmin in this order:

patient → tissue_source_site → cancer_type → sample → gene → variant_class → mutation
 phpMyAdmin import settings: Format=CSV, Columns separated=\t, Columns enclosed=" (double quote), Columns escaped=(blank), Lines terminated=auto

Then run these post-import SQL fixes in the phpMyAdmin SQL tab:

```
SET FOREIGN_KEY_CHECKS = 0;
INSERT INTO gene VALUES (0, 'UNKNOWN - entrez_gene_id 0 indicates no known gene ID');
DELETE FROM gene WHERE entrez_gene_id=1903 AND hugo_symbol='C9orf47';
DELETE FROM gene WHERE entrez_gene_id=23499 AND hugo_symbol='KIAA0754';
DELETE FROM gene WHERE entrez_gene_id=284697 AND hugo_symbol='KIAA1107';
DELETE FROM gene WHERE entrez_gene_id=399687 AND hugo_symbol='TIAF1';
SET FOREIGN_KEY_CHECKS = 1;
```

6. **Step 4** — Run SNP selection queries in phpMyAdmin SQL tab:

Open scripts/02_snp_selection_queries.sql and run the final query (HAVING >= 2 with mandatory dbsnp_rs). Export results as CSV.

7. **Step 5** — Build the Neo4j graph:

Open Neo4j Browser at <http://localhost:7474>. Paste and run scripts/03_neo4j_import.cypher. Verify with: MATCH (n)-[r]->(m) RETURN n,r,m

8. **Step 6** — Run API enrichment:

Update NEO4J_PASS on line 19 of scripts/04_api_enrichment.py, then:

```
python scripts/04_api_enrichment.py
```

11. Limitations and Future Work

11.1 Known Limitations

- Annotation freeze: MAF file uses GRCh37 annotation frozen circa 2014–2016. 744 genes have Entrez_Gene_Id=0 due to gene renaming or missing IDs at annotation time.
- No patient clinical data: The patient table contains only the barcode identifier. Age, sex, stage, and survival data were absent from the downloaded files.
- Single cancer type: All 512 samples are CCRCC. The cancer_type table has one row; the normalization benefit of that table only materializes when multiple cancer types are integrated.
- phpMyAdmin type drift: Six sample columns and three mutation columns have VARCHAR types in the live database instead of the intended numeric types due to phpMyAdmin CSV type inference. Numeric aggregation queries on those columns require explicit CAST.
- BAK1 low VAF: BAK1 p.R76W has mean VAF of 0.072, substantially lower than the other four selected SNPs (0.18–0.39), suggesting a subclonal rather than clonal event. Its biological significance as a driver is uncertain.
- HNF1B no rsID: HNF1B p.N302K has no dbSNP registration. dbSNP and ClinVar API calls return null for this SNP.

11.2 Future Work

- Integrate mRNA-seq expression data as an expression table linking sample and gene, enabling mutation-expression correlation queries.
- Remap 744 zero-ID genes using current NCBI Homo_sapiens.gene_info file to recover correct Entrez IDs for renamed genes.
- Extend Neo4j graph to all double-damaging missense SNPs (approximately 50 at HAVING >= 2) for a more complete PPI network.
- Lift over coordinates from GRCh37 to GRCh38 using UCSC LiftOver for integration with current annotation resources.

- Fix VARCHAR type deviations in the live database using ALTER TABLE statements to restore intended numeric types.

12. Expected Final Result

After successful execution of all scripts, the following should be verifiable:

12.1 MySQL verification

```
SELECT COUNT(*) FROM mutation;      -- expect 31073
SELECT COUNT(*) FROM sample;        -- expect 512
SELECT COUNT(*) FROM gene;          -- expect ~3800
```

12.2 Neo4j verification

```
MATCH (n)-[r]->(m) RETURN n, r, m
MATCH (s:SNP) RETURN s.hgvsp_short, s.grantham_score, s.clinvar_significance
MATCH (a:Gene)-[r:INTERACTS_WITH]-(b:Gene) RETURN a.hugo_symbol,
r.string_score, b.hugo_symbol
```

The final Neo4j graph contains 5 Gene nodes (VHL, MTOR, HNF1B, FNBP1L, BAK1), 5 SNP nodes enriched with Grantham scores and API data, 12 Sample nodes connected by HAS_MUTATION edges with VAF properties, and INTERACTS_WITH edges from the STRING PPI network.